

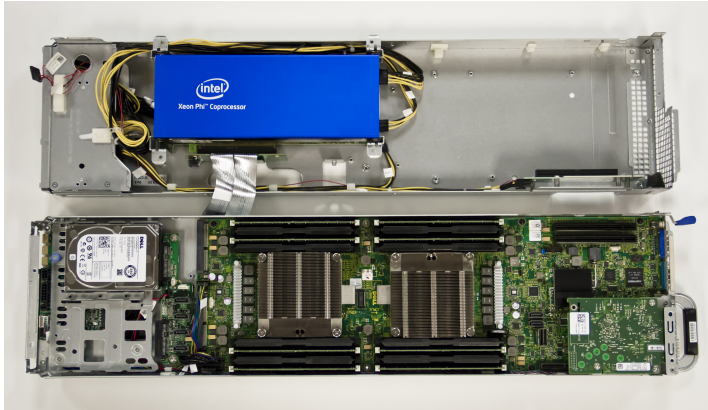
Fork-Join Model

Victor Eijkhout

2026 PCSE

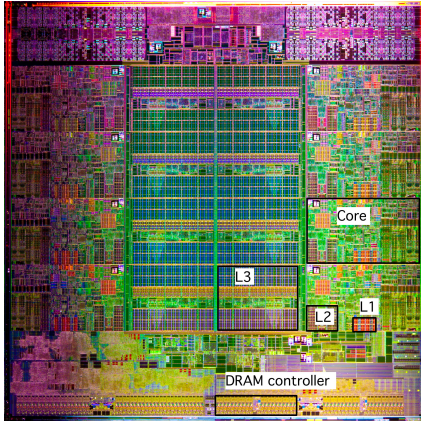
Computer architecture terminology

One cluster node:



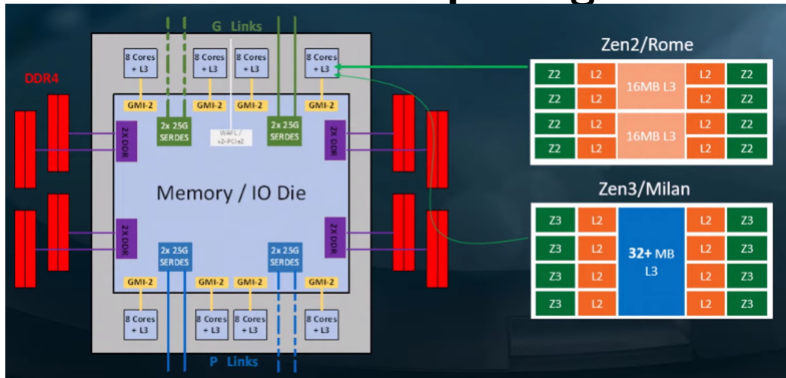
A node will have 1 or 2 or (sometimes) 4 'sockets': processor chips.
There may be a co-processor attached.

Structure of a socket



- Eight cores per socket, 16 per node.
- Access to DRAM, both this socket and other
- Shared L3 cache, private L2 and L1

Modern chip design



- Shared memory
- Shared caches
- Local caches
- 'cc-Numa': Cache-coherent Non-Uniform Memory Access

OpenMP history

- First standard 1997, Fortran directives
- Current standard 5.2 as of 2021

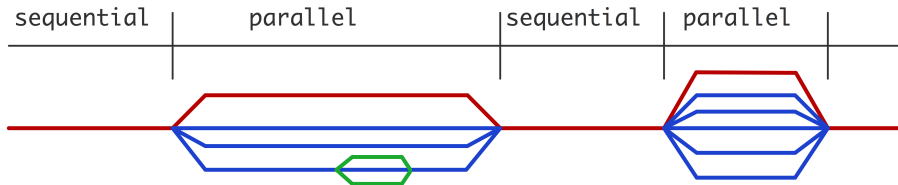
History of shared memory computers:

- Intel Core processors: 2006;
before that: 1980s systems from Alliant, Sequent, Denelcor;
SGI Origin with up to 1024 processors
supercomputers with multi-chip nodes (Intel Paragon, Cray T3D)

Threads

Process: stream of instructions

Thread: process can duplicate itself, same code, access to same data



The OS will place threads on different cores: parallel performance.

Note: threads are software. More threads than cores or fewer is allowed.

Ignore hyperthreads

To write an OpenMP program

```
1 #include "omp.h"
```

in C, and

```
1 use omp_lib
```

or

```
1 #include "omp_lib.h"
```

for Fortran.

To compile an OpenMP program

```
# gcc
gcc -o foo foo.c -fopenmp
# Intel compiler
icx -o foo foo.c -qopenmp
```

Run as ordinary program.

CMake

```
1 find_package(OpenMP)
2 # depending on language:
3 target_link_libraries( ${program} PUBLIC OpenMP::OpenMP_C )
4 target_link_libraries( ${program} PUBLIC OpenMP::OpenMP_CXX )
5 target_link_libraries( ${program} PUBLIC OpenMP::OpenMP_Fortran )
```

Let's try an example

How many cores do we have?

```
int omp_get_num_procs();
```

Compile and run

Query threads

```
int omp_get_num_threads();
```

Compile and run. Observe?

To run an OpenMP program

```
1 export OMP_NUM_THREADS=8
2 ./my_omp_program
```

Quick experiment:

```
1 for t in 1 2 4 8 12 16; do
2   OMP_NUM_THREADS=$t ./my_omp_program
3 done
```

Query threads'

Use a parallel region:

```
1 #pragma omp parallel
2 {
3   statements
4 }
```

or for Fortran:

```
1 !$omp parallel
2   statements
3 !$omp end parallel
```

Compile and run. Observe?

Query threads”

What happens if you write

```
1 int t;  
2 #pragma omp parallel  
3 {  
4     t = omp_get_thread_num();  
5     printf("%d\n",t);  
6 }
```

If you see a problem, can you suggest a solution?

Exercise 1 parallel

Write a 'hello world' program, where the print statement is in a parallel region. Compile and run.

Run your program with different values of the environment variable `OMP_NUM_THREADS`. If you know how many cores your machine has, can you set the value higher?

Exercise 2 parallel

Take the hello world program of exercise 1 and insert the above functions, before, in, and after the parallel region. What are your observations?

What happens if I press that button?

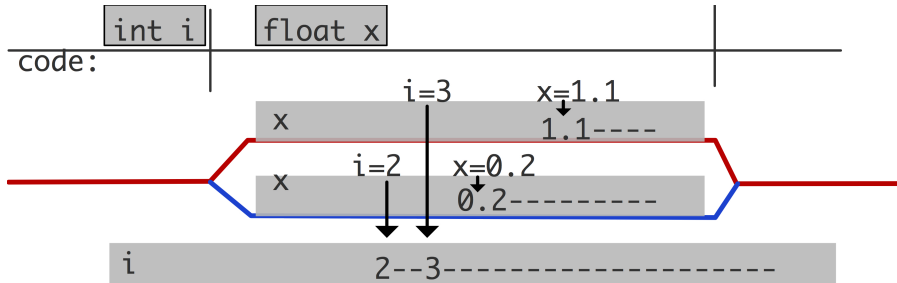
Who of you has tried setting the number of threads (much) larger than the number of cores? What happened?

Thread-local variables

```
1 {  
2   int x;  
3   #pragma omp parallel  
4   {  
5     double x;  
6     // do something with x  
7   }  
8 }
```

- A parallel region is a scope.
- Local variables are per thread.

Locality of variables



Thread control

- Environment variable `OMP_NUM_THREADS=4`
- Clause `num_threads(4)`
`#pragma omp parallel num_threads(4)`
usually not the best solution.
- Environment variable `OMP_THREAD_LIMIT=56`
(useful for nested parallelism)

Limits on parallelism

```
#pragma omp parallel if(N>50)
```

Can you think of an application?

I mean, what's the harm of doing one small loop inefficiently?

Threads and threads

- Threads are software, cores are hardware.
- The OS can move threads between cores: not a good idea for performance.
- Set: `export OMP_PROC_BIND=true` and you'll be good in most of the cases.
- Look up 'affinity' in the OMP standard for all the details.

Nested parallelism

```
1 int main() {  
2     ...  
3     #pragma omp parallel  
4     {  
5         ...  
6         func(...)  
7         ...  
8     }  
9 } // end of main
```

```
1 void func(...) {  
2     #pragma omp parallel  
3     {  
4         ...  
5     }  
6 }
```

Nesting control

By default, nested levels get only one thread.

Use:

```
1 void omp_set_max_active_levels(int);  
2 int omp_get_max_active_levels(void);
```

or

```
1 OMP_MAX_ACTIVE_LEVELS=3
```

(Deprecated: boolean switch *OMP_NESTED*)